SNES Assembly Adventure 03: Creating Your First Sprite

14 Aug 2018

Update May 2022: Fixed some errors in the example images which had some incorrect color values. Many thanks to the commenters for pointing this out. Also added additional information on how to prepare graphics for use on the SNES.

Update February 2022: All code examples from all articles in this series can now be found on Github in one repository

Welcome back, Adventurer. I hope the last time wasn't too hard on you. I admit it got a bit long (I strive to keep it under 2,500 words, the last one was 3,900). But we covered some important ground that will come in handy later.

In this article (should I start calling them dungeons instead? Levels? Quests? I'll think about it) we will go about things a bit differently. Since the last article was heavy on the theory, I want to keep this one a bit more practical.

I promised you in the last article that you will create a sprite to display later on the SNES, so in this article, we will

- discuss the graphics format of the SNES
- create a simple sprite by hand

And in the next article, we will use that sprite and display it on the SNES. Let's get started!



Your Development Environment Badge has evolved to the First Opcodes Badge!

Intertwined and Planar Graphic Formats

The SNES relies on a technique called *(indirect) color indexing* (in fact, the NES and Game Boy used this technique too, as we will see shortly). Instead of saving the color of a certain pixel, the index number of a color is saved that references a color inside a palette.

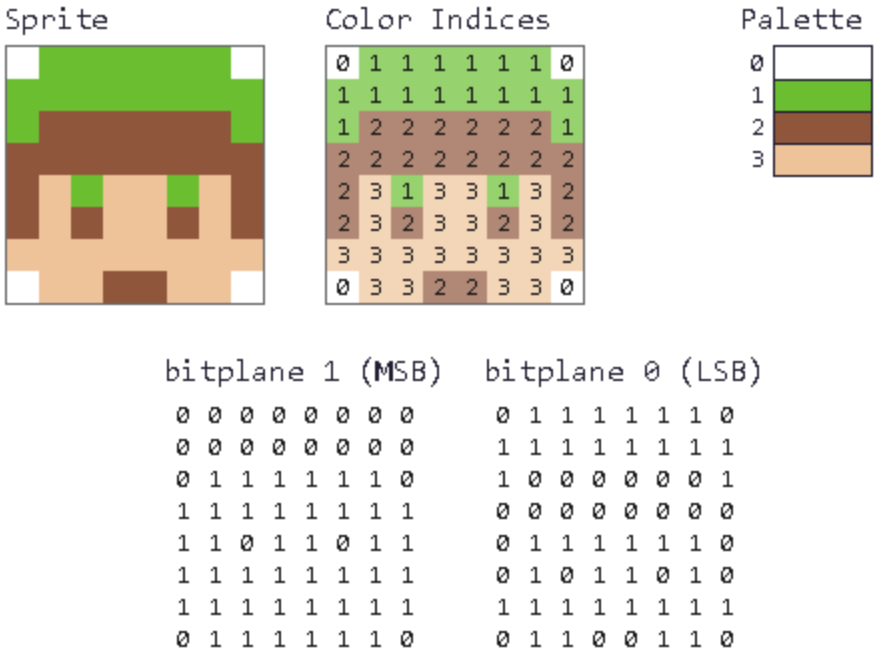
Let's look at a simple 8 x 8 example. This first image is in *1 bit per pixel* format, or 1bpp:



1 bit per pixel format

Each pixel holds 1 bit. So this means, a pixel can have two colors, either color 0 or color 1. On the left, you see the sprite. To the right of it, you see the *color index* assigned to each pixel (i.e., the index of the color the pixel will have). The numbers under *Mem.* are the indices of the corresponding pixel row translated to hexadecimal. And finally, the palette that shows which index represents which color.

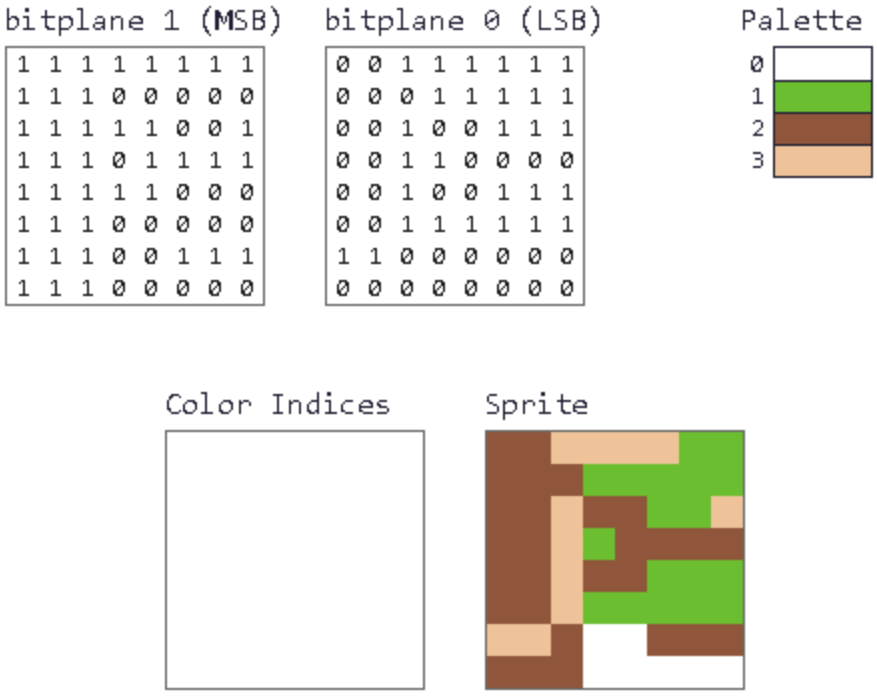
Now, only two colors are nice for certain art styles, but to relive the glory of 16-bit graphics we need a bit more (no pun intended). Let's extend the format to *2 bits per pixel* (i.e., 2bpp). Having two bits per pixel allows us to assign four different colors to each pixel (remember how binary numbers work?). Here is an example:



2 bit per pixel format

Let's introduce the concept of *bitplanes*. Since we now need two bits to store the information for one pixel, we need to think about how to store those bits in memory. Each byte in a bitplane represents one row of the sprite. Each bitplane holds one bit of the color index of the corresponding pixel. So to get the color index of a specific pixel we need to combine the data of each bitplane. For example, to get the color index of the first pixel of the first row (the pixel on the upper far-left), we look at the first bit of the first row of each bitplane and combine them to form a 2-bit number.

Let's do an example:



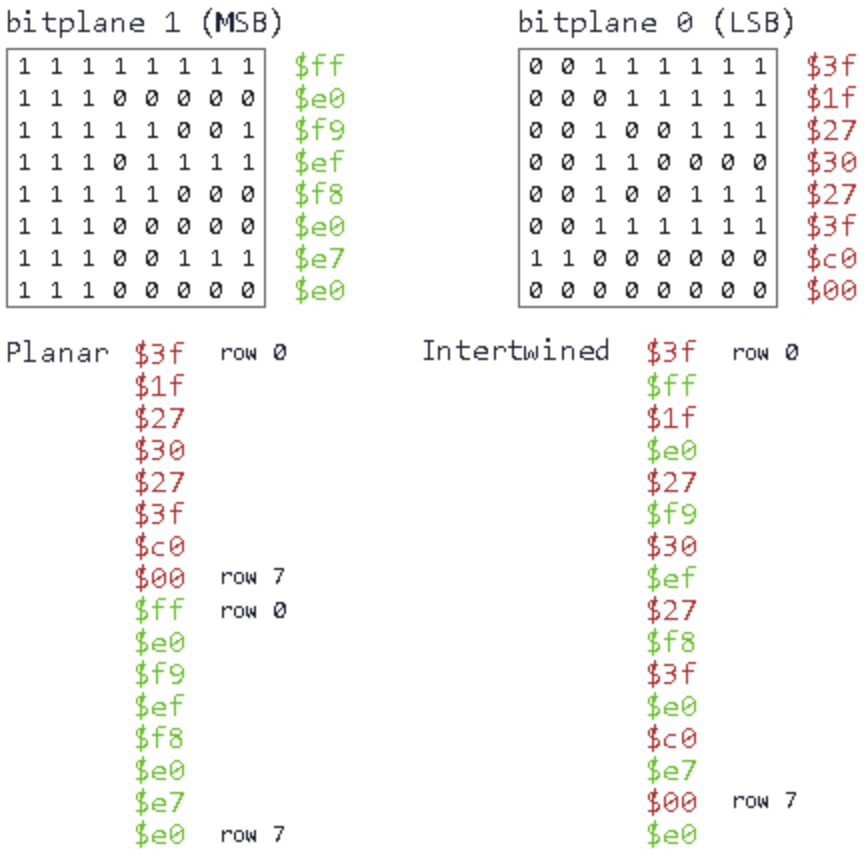
Converting a sprite into 2bpp

This can be hard to wrap one's head around at first. I suggest you take the data from the gif above and try to complete the sprite by hand on paper. Yes, it's tedious. But when we talk about the architecture of the SNES in a later article, a lot of the things described here will be easier to understand. Most of the graphical limitations of the SNES stem from the way it stores and processes graphics data.

Now is a good point to talk about the section title, *intertwined* and *planar* data storage. Once we have converted our graphics to a format the SNES understands. There are two ways to store the data in memory:

- **Planar**: Each row (i.e., byte) of a bitplane is stored consecutively, starting with the top row of the lowest bitplane; then the same is repeated with each row of the next bitplane
- **Intertwined**: Each pair of bytes consecutively stored represents a row of the sprite with the byte from the lower bitplane being stored first

This again might sound confusing, so let's try to clarify this with another picture:



Intertwined and planar graphics

On top, you have the two bitplanes and their hexadecimal equal. Below you see the graphics data stored once in planar and once in intertwined format. In both cases, The bytes are stored in consecutive memory locations from top to bottom.

Why does the distinction matter? Nintendo has used both formats in their consoles. While the NES stored graphics data in planar format, the Game Boy used the intertwined format. Both consoles could display sprites with a maximum of four colors (why they used different formats on each console is anyone's guess; the overall architectures of both consoles aren't that different, to it probably came down to a factor like performance or system design).

So which format does the SNES use? The answer is both.

I know, more confusion. But before I explain that, there is an important piece of information to add: While I simply used white as the color 0 of the palette, this isn't entirely correct. The first color of a palette has a special function. The SNES considers the first color in any palette as *transparent* and will not render it. So any pixel that has the color index 0 will not be displayed. We will see this in action shortly.

Now, back to planar and intertwined. I told you the SNES uses both formats. Why? The SNES (unlike the NES and Game Boy) can actually switch between different bits-per-pixel formats. In fact, the SNES supports a total of five bit formats: 2bpp, 3bpp, 4bpp, 8bpp, and the special Mode 7 format (this is the ominous mode used in some games to simulate some kind of pseudo-3D rendering, e.g. the Final Fantasy VI opening sequence, Mario Kart, or F-Zero; I will cover Mode 7, but that is -sadly- still some way off).

You already know 1bpp and 2bpp. Extending 2bpp to 3bpp and 4bpp isn't hard if you understand how color indexing works by now. You simply add the extra bitplanes needed. So 3bpp will let you use a total of eight colors, 4bpp a total of 16 colors. The two most frequently used formats are 2bpp (HUD elements and background) and 4bpp (backgrounds and sprites).

This concludes the section on the color-indexed graphics format. If you feel overwhelmed by it, two excellent resources explain this in more detail. You will find links to them at the end of this article in the *Links and References* section.

Let's put this new knowledge to good use and actually create a 16 x 16 sprite (which is simply four 8 x 8 sprites combined) in 4bpp format by hand. Finally, we get practical, as promised!

(If you're wondering, "Wait! I have now all the color indices, but where are the *colors themselves*?" You are correct, we haven't covered that yet. This will happen in the next section.)

Handcrafting a Sprite

Before we start, let me state that this is **not** the best way to create graphic assets for the SNES. There are a lot of tools on the web that will automatically convert assets in jpg, png, bmp, or any other format to 2bpp/4bpp for you. I will link some common tools in the *Links and Reference* section at the end of this article.

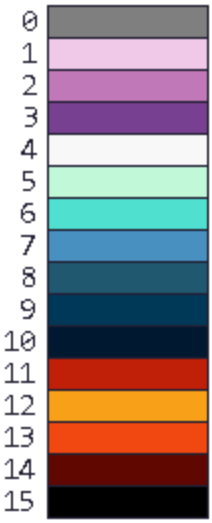
Since the victory of Sonicfox at Evo 2018 (as of the writing of this article in August 2018) is all the rage right now, will recreate the face of Super Kai from *Dragon Ball Z: Super Butouden 3*. I am a horrible pixel artist, so I won't take you through all steps of creating pixel art, there are more than enough tutorials and guides around on the web. So, here are the finished sprites:



Totally rad sprites

Since we're going to use a 4bpp format, I can use up to 16 colors (remember to reserve color 0 for transparency). Let's start by creating the color indices:

0	0	6	0	7	6	7	5	6	5	6	5	4	4	4	5
0	0	7	6	7	6	7	6	7	6	5	5	5	5	4	4
0	0	0	7	7	7	7	6	7	6	6	6	5	5	5	4
0	0	6	6	7	7	3	3	7	6	5	5	4	4	5	5
0	0	7	6	7	3	3	2	2	6	7	7	6	5	4	5
0	0	0	7	3	3	2	2	1	1	7	2	7	6	5	4
2	2	2	2	3	3	2	2	2	1	1	1	2	2	6	4
3	3	3	3	2	2	3	1	1	2	1	1	1	2	0	4
0	2	3	2	3	3	2	2	1	1	1	1	1	2	1	2
0	0	2	3	3	2	2	3	3	1	1	1	3	2	3	2
0	0	0	2	2	3	2	4	2	3	1	3	4	2	2	0
0	0	0	0	3	3	2	2	1	1	2	1	1	2	0	0
0	0	0	c	0	0	3	2	1	3	3	1	2	c	0	0
0	0	0	0	0	e	b	3	1	1	1	2	0	0	0	0
b	b	8	9	9	a	8	b	3	3	3	d	0	0	0	0
8	b	b	8	9	9	9	8	b	b	d	7	8	7	b	0



Color indices of totally rad sprites

No surprises here. There is no need to use all 16 colors, so the last color of the palette is simply set to black. Since we can use any color for transparency, I simply chose a gray tone.

Now, let's translate this by hand with the help of a hex editor. I prefer hyx, but you can use any editor you see fit. I'll link some down in the *Links and References* section. Mind again that I'm doing this only for educational and demonstration purposes. When creating your own game assets you can, of course, rely on (automated) tools to do this step for you.

I will take the first row of the first sprite (upper-left) and work my way from the left-most pixel to the right. Since we're using 4bpp, we now have 4 bitplanes: bitplane 0, 1, 2, and 3. Now, do we store the bitplanes in planar or intertwined format? The answer is both.

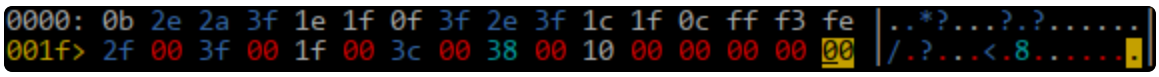
Here is how we need to store the graphics data in memory so the SNES can read it:

- Start with the first row of the sprite
- Take the byte that represents bitplane 0 and store it in memory
- Now take the byte that represents bitplane 1 and store it in the next memory location
- Repeat this for all eight rows of the sprite



Converting bitplane 0 and 1 of the first row of the first sprite

So, this is obviously intertwined. Now we have stored bitplanes 0 and 1 in memory, so the file is now 16 bytes long (1 byte times 8 rows times 2 bitplanes equals 16 bytes). But what happens with bitplanes 2 and 3? Easy, rinse, and repeat! We take the exact same steps as above just with bitplane 2 and 3 this time. Here is the finished first tile in my hex editor:



The sprite converted to 4bpp

Now, repeat this for the other three tiles. One tile needs 32 bytes, so your final file should have a size of 128 bytes (crazy, I know! The favicon in your browser is larger than this). Save this as `Sprites.vra` in `/assemblyadventure/lesson03/` or wherever you prefer to keep your files. Mind that the file extension is arbitrary; I chose `.vra` to represent VRAM (which stands for *Video RAM*). Some choose `.rom` or `.bin`, it doesn't really matter. Just keep it consistent. You will need those files in the next article when we write the code to display them on the SNES.

We have converted our four tiles into a 4bpp format the SNES can read. Now the only thing missing is the palette, i.e., color data.

SNES Palettes

Since we're using color indexing, we need to store the actual color values somewhere. The sprite data and color data are stored in two separate files. Why that is will be part of a later article on the architecture of the SNES.

You're probably familiar with the way computers store color information. The most widely used format is RGB888 (or RGB8888 if you add transparency). You store the amount of red, green, and blue that make up a color. The color components mixed together then yield the final color on the screen.

Unlike the NES and Game Boy (both had fixed palettes developers are forced to use), the SNES can actually display any color that fits into its BGR555 format. Mind the BGR part, the SNES stores the color components in reversed order.

So, how do we convert an RGB888 color to BGR555? We simply take the five most significant bits of each color component (i.e., we discard the three least significant bits) and reverse their order:



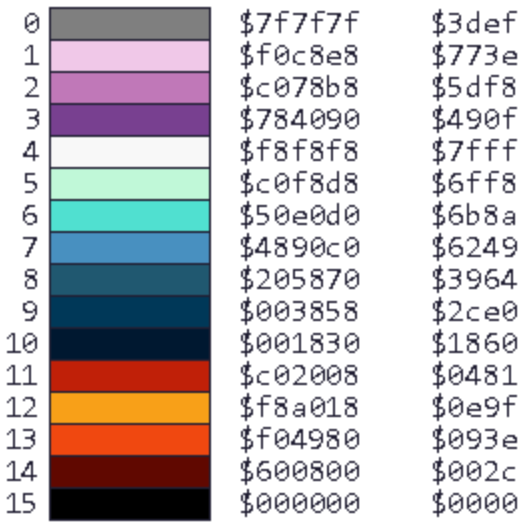
Converting RGB888 to BGR555

Mind we only use 15 out of the 16 bits of a two-byte number, make sure to always clear the most significant bit to 0. Try and see if you can convert the 16 colors of the palette we used for our

sprites above. Go! Do it! Don't cheat!

Okay, are you done yet? And you didn't cheat?

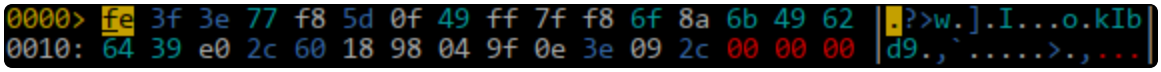
Here is the solution:



Complete palette converted to BGR555

Since each color takes two bytes to store, a full (4bpp) palette will take up 32 bytes. I haven't told you that yet, but the 65816 is a little-endian machine. This means, that if you need to store information that is longer than one byte, the store the lower byte first, then the higher byte.

Let me demonstrate this again in my trusty hex editor:



The finished palette file in hex

I took the converted data from the picture above and wrote the first the lower, then the higher byte to the file. Save this file as `SpriteColors.pal`. I again chose an arbitrary file extension, `.pal` for a palette.

So, now we have our graphics data in `Sprites.vra` and the color palette in `SpriteColors.pal` ready to use. Congratulations, you have created your first SNES graphics!

Again, the process I described here is definitely *not* the way to go about when creating assets for your game. But it is essential to understand the underlying rules and formats. There are a few tools specially made for SNES development, I'll link some at the end in the *Links and References* section.

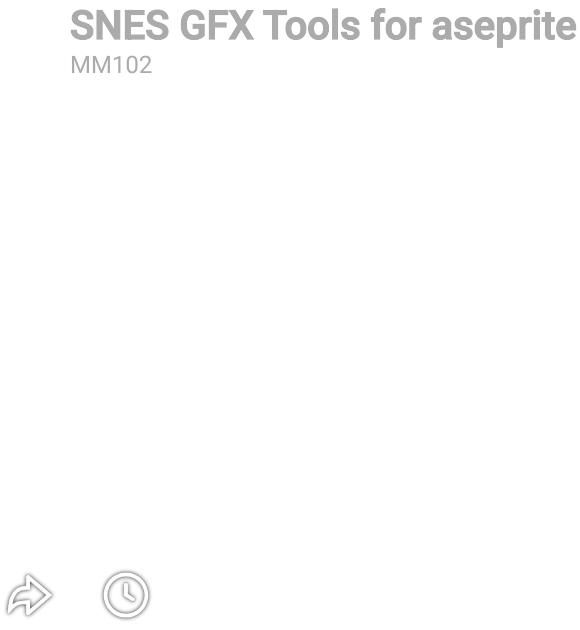
Graphics Tools

I've written a simple conversion tool a while back. But I'm no longer actively working on it (Qt is just too weird for me). So here are some suggestions for other tools you can use.

Personally, I use **Pro Motion NG** by Cosmigo for my pixel art (if you can call it that...) as it also works great with graphic tablets. There's a free version you can download here. It is also available on Steam. As of May 2022, Steam users can access the beta of version 8 (the current stable version in May 2022 is 7.2.7). Highly recommended.

A great open source alternative is **Aseprite**. I use Aseprite before I switched to Pro Motion NG. In fact, all of the title cards, images, etc. you see in SNES Assembly Adventure articles were created with Aseprite originally. You can either download the source code on Github, purchase it from their website, or get it on Steam.

Aseprite also offers great community support. There is even this excellent tool/script specific for SNES developers:



Find the download instructions in the video's description.

Here's an unordered list of other good tools:

- GraphX2 is another great bitmap graphics tool with a retro feel to it
- SuperFamiconv by Optiroc is a command line tool that will allow converting image files
- Nesdoug's allows you to create and edit tiles and tilemaps

Conclusion

After the last programming theory-heavy article, this article explained (a bit more practical) how the graphics format on the SNES works (in fact, most retro consoles like the NES, Game Boy, Genesis/Mega Drive, etc. use this format in some variation). You created your own first sprites and palette by hand. In the next article, we will take these sprites and palette and display them on the SNES. So finally you will put your theoretical knowledge to use!

As always, comments and suggestions are always welcome. Please use the comment function below.

Links and References

- If you struggle with the graphics formats described here, I highly recommend you check out these two links:
  - This video explains how bitplanes and palettes work in great detail. His other videos are fantastic too, definitely subscribe to his channel!
  - Another great explanation can be found here. Don't be put off by the design, it's from a simpler time. This also includes information on a lot of other consoles.
  - Mega Cat Studios has a great guide on SNES graphics. They also make awesome homebrews.
- Here are some tools for converting graphics to a SNES format:
  - YY-CHR is the preferred tool of many ROM hackers (yes, there are still *active* geocities sites). Check out this tutorial on how to use it.
  - ~~I have written my own tool. It will take a single file and automatically convert it to a 4bpp and palette file. It's in a very early stage and by no end finished yet. Use at your own risk.~~ I'm no longer actively working on this.
  - The awesome ROM hackers at smwcentral.net have an extensive list of tools you can use.
- Here are some alternative hex editors if you don't like working solely on the command line:
  - HxD is a good hex editor for Windows users.
  - hexfiend should make the OS X user happy.
  - wmMEdit is a cross-platform hex editor with a nice GUI.

- All SNES Assembly Adventure code examples of this series on Github

ALSO ON MACHINE CODE CONSTRUCTION YARD

SNES Assembly Adventure 07: Direct …

7 years ago · 26 comments

I do assembly programming on old machines and boat building.

A CMake Template for Game Boy Development

3 years ago · 1 comment

I do assembly programming on old machines and boat building.

SNES Assembly Adventure 05: …

8 years ago · 8 comments

I do assembly programming on old machines and boat building.

SNES Assem Adventure 0

8 years ago · 6 c

I do assembly on old machin building.